Matryoshka Representation Learning 俄罗斯套娃表示学习

研究背景

1. Q：为什么做MRL，MRL是在什么环境下产生的?

   A：现在又这样的一个数据集，转成向量之后可能会完成不同的下游任务，然而任务有难易区分，高维度的向量表示来做简单的任务会增加运算负担，低维度向量做精度要求高的任务无法达到任务要求，因而现在需要更加高效的对于一套数据集训练能够产生不同维度的向量

2. Q：分类和检索对于比较大的数据库会很慢，原因和解决办法

   A：特征向量维度d很大，会直接导致计算变得很慢 -- 之前的解决办法：降维 哈希 特征选择（选择重要的特征进行计算） 缺点就是会牺牲精度

   整个库里面进行检索，暴力检索的话会是大概O(dN)的时间复杂度 -- 之前的解决办法：ANNS 可以将复杂度降到O(dlogN) 缺点就是还是没有解决复杂度仍然和d成线性增长

```
MRL对于第一个问题：因为MRL产生的向量任何的前m维度都可以直接使用的，所以可以根据需要选择合适的向量维度，而且MRL训练的时候会把各个损失都下降回去，因此不会牺牲精度，前m维度仍然是精准的。

MRL对于第二个问题：MRL可以有效的减少d，从而环节复杂度随d线性增长的问题。
```

3. Q：同网络多容量网络

   A：同一个网络，不同的容量，分别产生不同维度的向量，训练过程如下：

```
先训练大网络L，然后冻住L的参数，
然后训练小网络S，然后冻住S的参数，
然后训练中等大小的网络M，然后冻住M，
最后都解冻,放在一起,一起训练一次
```

问题1：各个大小的网络的参数的损失并不相同，梯度下降的方向不同，很难训稳
问题2：搞出来了很多的参数，其实没有节省任何的内存

4. Q：MRL的定位是什么？ A：类似于插件，可以和其他的各种方法共同使用并不冲突。这是一种训练方法。

MRL理论解释

1. 主要把MRL损失计算的公式弄明白就可以了

   复杂公式（MRL）：

$$\boxed{\min_{\{W^{(m)}\}_{m\in\mathcal{M}},\theta_F} \frac{1}{N}\sum_{i\in[N]}\sum_{m\in\mathcal{M}} c_m \cdot \mathcal{L}(W^{(m)}\cdot F(x_i;\theta_F);y_i)}$$

   c_m在这个计算中等于1　$W^{(m)}$正常来说是每个维度单独计算一个，实际做的时候是训练一个完整的，根据多少维取多大矩阵，也就是大矩阵的切片$W^{(m)} = W_{1:m}$ 这个就是MRL-E 普通的MRL就是每个维度一个权重矩阵$W^{(m)}$

   m 是$\mathcal{M}$中的元素，$\mathcal{M}$是选取的进行计算监督损失的维度的集合，就比如他们在一个2048维度的训练中选取的$\mathcal{M}$ = {8,16,⋯，1024，2048}

简单公式(MRL-E)：

$$\boxed{\min_{\{W_{1:m}\}_{m\in\mathcal{M}},\theta_F} \frac{1}{N}\sum_{i\in[N]}\sum_{m\in\mathcal{M}} \mathcal{L}(W_{1:m}\cdot F(x_i;\theta_F);y_i)}$$

F是代表模型对数据已经理解好了,输出是一个d*1的向量，W是一个L*d的向量，W是把理解好的内容映射到对应任务对应的输出结果，和y_i比较通过$\mathcal{L}$计算$loss$

其中L是例如分类任务有L类，$W \cdot F$输出是L*1的向量，代表是各个类别的可能

$$\boxed{\mathcal{L}(z,y) = -log(\frac{e^{z_y}}{\sum_{j=1}^{L} e^{z_j}})}$$

log里面是对应的softmax概率就是代表模型认为这是哪一类的概率 ,z是向量，y是实际的标签，z_y代表z中对应标签的也就是正确的概率

$$\boxed{softmax(z,y) = \frac{e^{z_y}}{\sum_{j=1}^{L} e^{z_j}}}$$

为什么要用e?

```
防止负数
拉开梯度
容易求导 方便训练
```

$$\boxed{loss_{total} = \frac{1}{N} \sum_{i \in [N]} \sum_{m \in \mathcal{M}} \mathcal{L}(W_{1:m} \cdot F(x_i;\theta_F); y_i)}$$

然后用$loss_{total}$对θ_F和W求导，对θ_F和W进行更新，更新规则如下：

$$\boxed{W \leftarrow W - \alpha \cdot \nabla_W \mathcal{L}}$$

$$\boxed{\theta_F \leftarrow \theta_F - \alpha \cdot \nabla_{\theta_F} \mathcal{L}}$$

α学习率，每一次步子迈多大

$\nabla_x y$决定步子往哪个方向迈，因为x不一定是一个数，可能是向量啊矩阵啊等等

这个训练的目的就是为了让这个loss尽可能的小

MRL应用

1. ResNet50输出的是2048维的特征表示，而ViT-B/16和BERT-Base对每个数据点输出768维的特征表示

   实验中采取的$\mathcal{M}$ 分别是

   1. $\mathcal{M}$={8,16,32,64,128,256,512,1024,2048}
   2. $\mathcal{M}$={12,24,48,96,192,384,768}

2. 将MRL & MRL-E和下面的几种方法进行对比，结果是MRL&MRL-E更好一点

   1. 低维固定特征表示FF 要多少维就用多少维
   2. 降维方法SVD 训练高维 用低维就用数学方法降维就好了
   3. 子网络方法（可瘦身网络）Slim. Net 网络嵌套 训练一个大网络里面包含很多小一层的子网络，推理的时候可以用小网络，也可以用大网络
   4. 最高容量FF模型的随机特征选取 Rand. FS Rand. LP 这两个就是纯随机挑选，这个纯随机 就垫底的baseline 最垃圾的

      1. Rand. FS是随机调几维
      2. Rand. LP是用一个随机矩阵随机映射到对应的维度上

3. MRL只显式优化了其中O($\log d$)个向量，但是其中的向量通过插值的效果也都很好

4. MRL实现了自适应分类和推理 自适应实现流程如下：

   1. 提前会设定一个softmax概率阈值，这个阈值代表大模型认为自己的判断是正确的自信程度，
   2. 如果超过阈值，就用这个维度了
   3. 如果没有超过阈值，就增加一些维度，然后再进行预测

   实现了：简单题快做，难题细做

深度讨论和消融

1. 选粒度最好是出现信息瓶颈来选更合适，不要均匀的粒度，实验证明了前者更好一些

结论

1. 平均用于分类和推理的维度比固定少了14倍，准确率相当
2. 运算时间相较于基线模型快了14倍